

Djahiri

Livrable 2 Rapport d'Avancement : Conception du Projet Djahiri

Présenté par:
Yves Roland DEBO
et Zakariae BOUAMAMA

Encadré par:
M. ZOUARI, M. YAACOUB et M. ELKETROUSSI

Table des matières

1	Architecture Fonctionnelle	2
1.1	Présentation et Objectifs	2
1.2	Cycle de vie d'un signalement	2
1.2.1	Processus de soumission et modération	2
1.3	Interfaces de la solution	3
1.3.1	Application Mobile (Citoyens)	3
1.3.2	Panel Administration (Web)	4
2	Conception Logicielle	6
2.1	Architecture Globale (Modèle C4)	6
2.2	Architecture Hexagonale et CQRS	7
2.2.1	Découpage par couches	7
2.2.2	Isolation par les Ports et Inversion de Dépendance	7
2.2.3	Convergence avec le pattern CQRS	7
3	Modélisation des Données	9
3.1	Schéma de la base de données	9
3.1.1	Collection <code>users</code>	9
3.1.2	Collection <code>reports</code>	9
3.1.3	Collection <code>badges</code>	9
3.2	Justification du modèle Document	10
4	Infrastructure et Déploiement	11
4.1	Stratégie de Containerisation	11
4.1.1	Isolation des services	11
4.1.2	Reverse Proxy et Sécurité	11
4.2	Pipeline CI/CD et Environnements	11
4.2.1	Gestion des environnements	11
4.2.2	Flux de déploiement continu	12
4.3	Persistance et Cloud	12

Chapitre 1

Architecture Fonctionnelle

Le projet **Djahiri** s'inscrit dans une démarche de *civic tech* visant à transformer la gestion des incidents urbains en Côte d'Ivoire. Contrairement aux solutions traditionnelles descendantes, Djahiri place l'engagement citoyen au cœur du dispositif grâce à une approche communautaire et ludique.

1.1 Présentation et Objectifs

L'objectif principal est de fournir un outil mobile robuste permettant aux citoyens de signaler des incivilités (dépôts sauvages, voiries dégradées, problèmes de propreté) en temps réel. Les objectifs fonctionnels se déclinent comme suit :

- **Signalement géolocalisé** : Capture de photo et récupération automatique des coordonnées GPS.
- **Suivi en temps réel** : Consultation de l'état d'avancement des signalements (En attente, Validé, Rejeté, Résolu).
- **Gamification** : Valorisation de l'action citoyenne par l'octroi de points de fidélité et de badges de reconnaissance.
- **Sensibilisation** : Diffusion de messages de prévention et de bonnes pratiques civiques.

1.2 Cycle de vie d'un signalement

Le processus métier de Djahiri repose sur une interaction étroite entre le citoyen et l'administration (modérateurs).

1.2.1 Processus de soumission et modération

Lorsqu'un citoyen identifie un problème, il suit un parcours structuré :

1. **Capture** : Prise d'une photographie via l'application mobile (l'upload de fichiers existants est restreint pour garantir l'authenticité).
2. **Qualification** : Sélection du type d'incivilité et ajout d'une description textuelle.
3. **Enquête** : Le signalement passe au statut "En attente". Un administrateur effectue une vérification pour s'assurer de la véracité des faits.
4. **Validation/Rejet** : L'administrateur valide le signalement (octroi de 10 points) ou le rejette avec un motif.

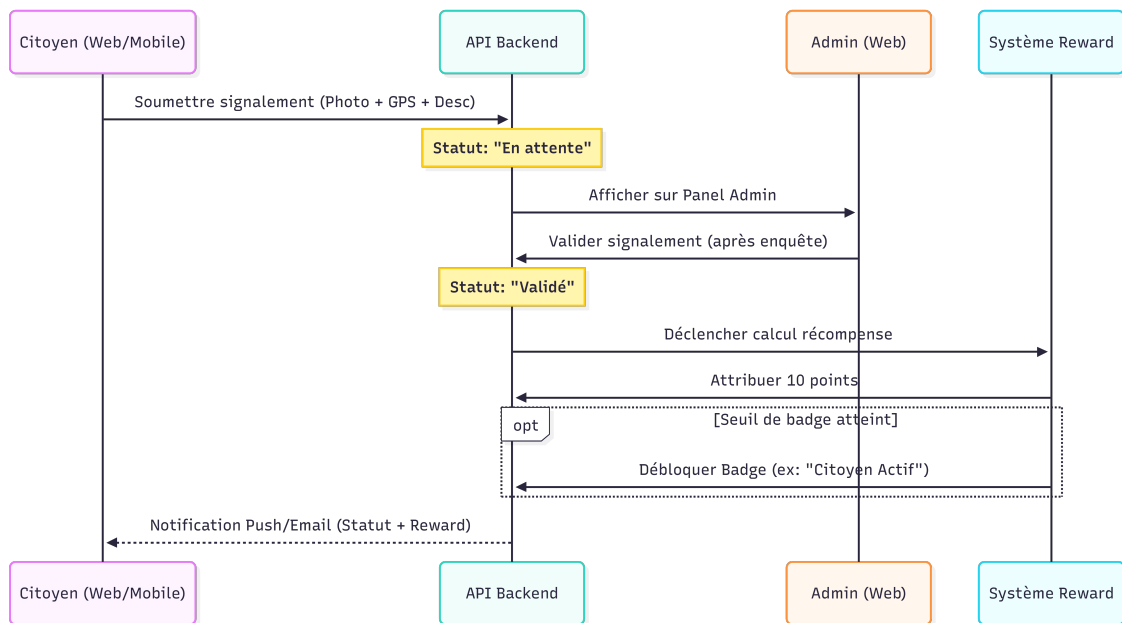


FIGURE 1.1 – Diagramme de séquence du cycle de vie d'un signalement

1.3 Interfaces de la solution

La solution Djahiri est composée d'une application mobile (Flutter) et d'une application web (ReactJS) pour les citoyens et d'un panel d'administration Web (ReactJS) pour la gestion et la modération.

1.3.1 Application Mobile (Citoyens)

L'interface mobile est conçue pour être intuitive, permettant une remontée d'information en quelques clics.

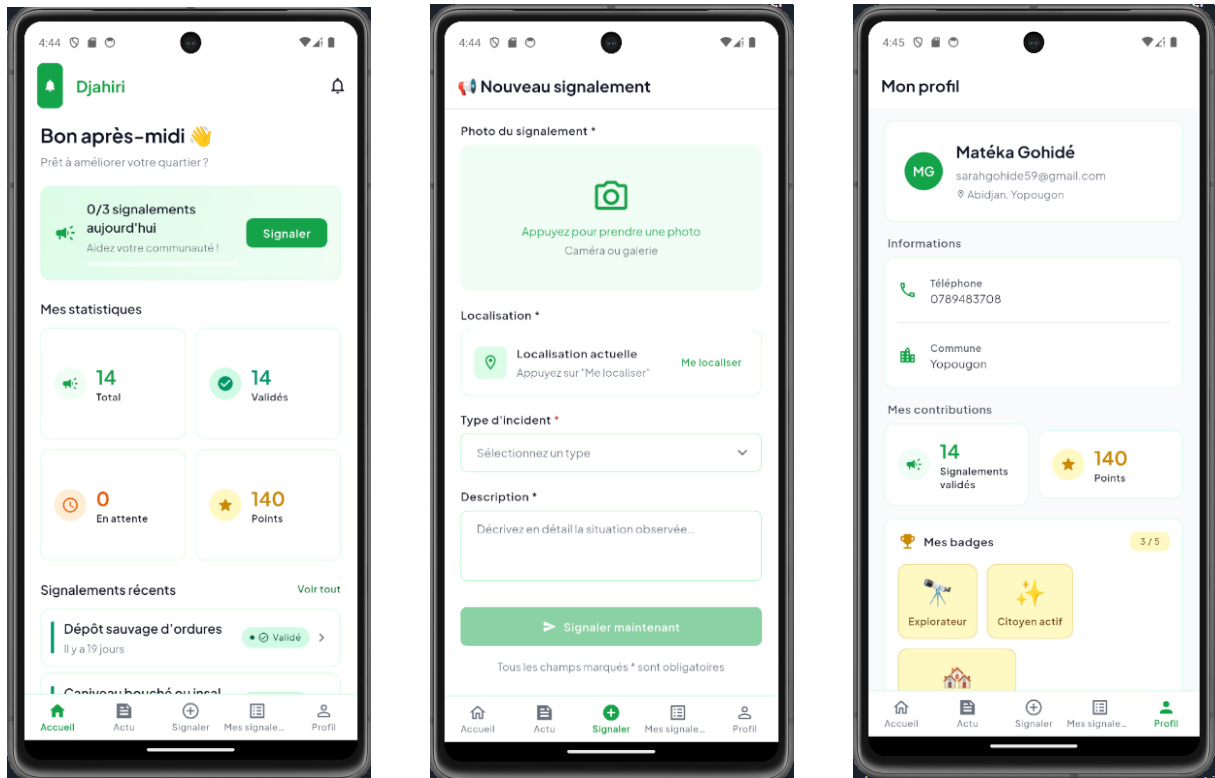


FIGURE 1.2 – Interfaces de l'application mobile Flutter

1.3.2 Panel Administration (Web)

La version Web permet aux administrateurs de visualiser l'ensemble des signalements sur une carte, de gérer les utilisateurs et de valider les enquêtes en cours.

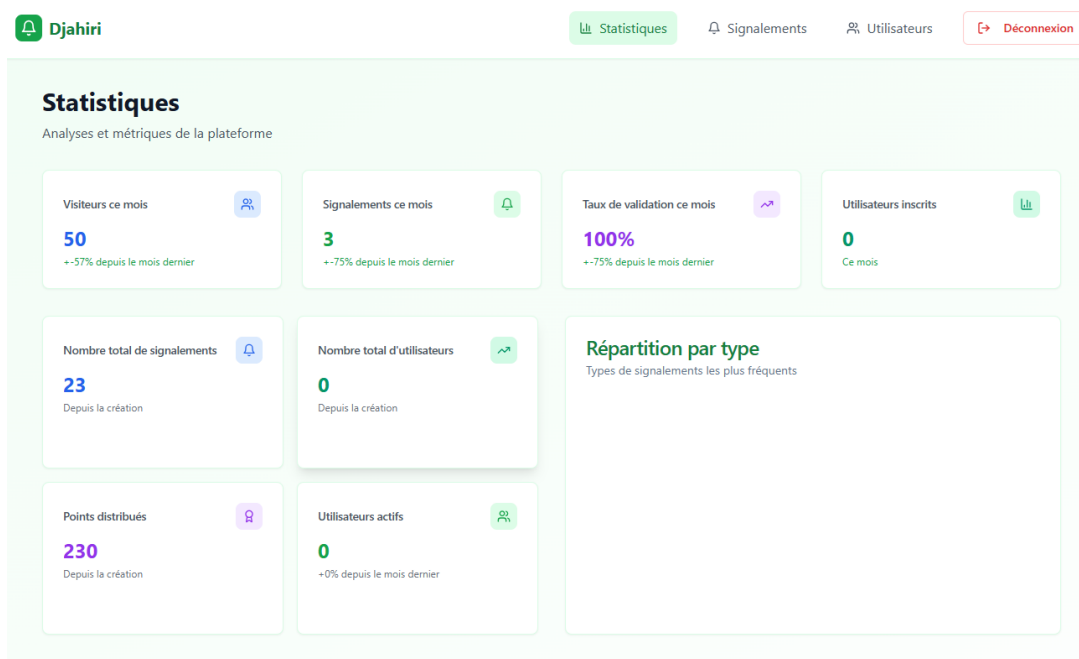


FIGURE 1.3 – Tableau de bord de l'interface d'administration ReactJS

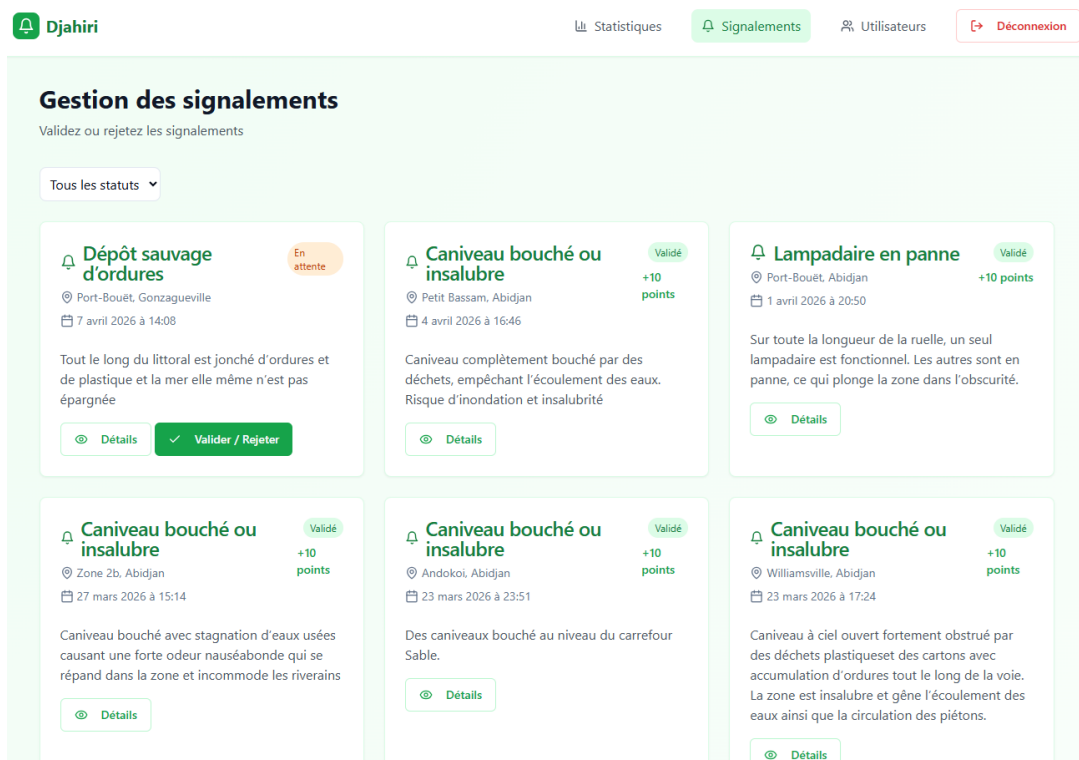


FIGURE 1.4 – Page de modération des signalements des citoyens

Chapitre 2

Conception Logicielle

La conception logicielle de **Djahiri** repose sur des principes d'ingénierie avancés visant à garantir la maintenabilité, la testabilité et l'évolutivité du système.

2.1 Architecture Globale (Modèle C4)

Le système adopte une architecture hybride. Le backend custom (Spring Boot) centralise la logique métier et la persistance (MongoDB Atlas), tandis que des services managés (Firebase) gèrent les notifications temps réel.

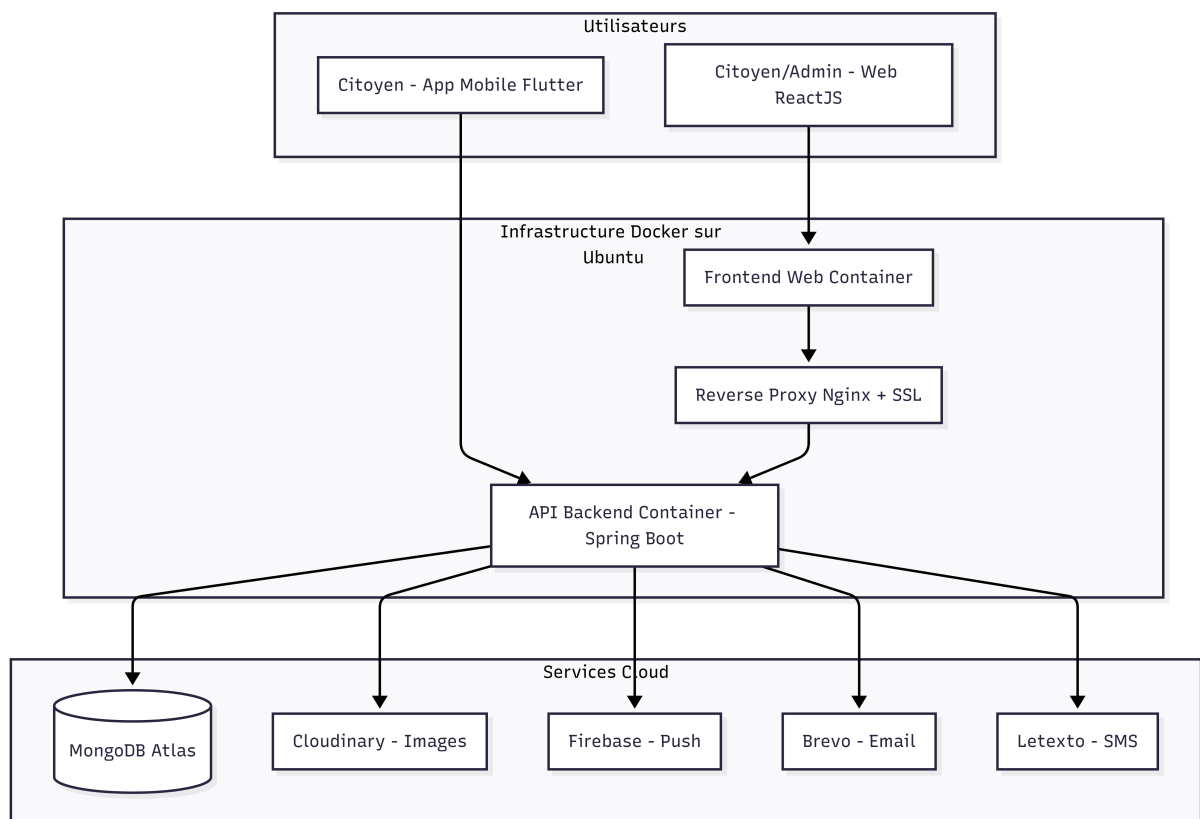


FIGURE 2.1 – Architecture globale de la solution

2.2 Architecture Hexagonale et CQRS

L'architecture est structurée physiquement pour séparer les responsabilités de commande (écriture) et de requête (lecture).

2.2.1 Découpage par couches

L'organisation du code source reflète une séparation stricte des préoccupations :

- **Couche Domaine** (`domain/`) : Le cœur du système. Il contient la logique métier (`usecases/`), les interfaces (`ports/`), les modèles métier (`request/`) et les événements (`events/`). Cette couche est isolée de toute dépendance technologique (Spring, MongoDB).
- **Couche Présentation** (`rest/`) : Gère les interactions HTTP via les contrôleurs et les modèles d'échange (`dto/`).
- **Couche Persistance** (`database/`) : Implémente l'accès aux données MongoDB à travers des `documents/` et des `adapters/`.
- **Couche Infrastructure** : Gère les services tiers (Firebase, Cloudinary).
- **Couche Sécurité** : Gère la configuration JWT.

2.2.2 Isolation par les Ports et Inversion de Dépendance

Le domaine définit des interfaces de type **Port** (ex : `ReportQueryPort`). Les implémentations concrètes (**Adapters**) se situent à l'extérieur du domaine. C'est le framework Spring qui injecte les adaptateurs au runtime, permettant au domaine de rester agnostique vis-à-vis des solutions de stockage ou d'infrastructure.

2.2.3 Convergence avec le pattern CQRS

L'utilisation des répertoires `in` et `out` matérialise le pattern CQRS de bout en bout :

- **in (Commandes)** : Regroupe les flux modifiant l'état du système (ex : soumission d'un signalement). Il s'appuie sur les `dto/in` et les ports d'entrée pilotant les Use Cases.
- **out (Requêtes)** : Optimise la lecture. Les adaptateurs `out` effectuent des requêtes MongoDB ciblées et retournent des `dto/out` optimisés pour l'affichage, sans charger inutilement les entités métier complètes.

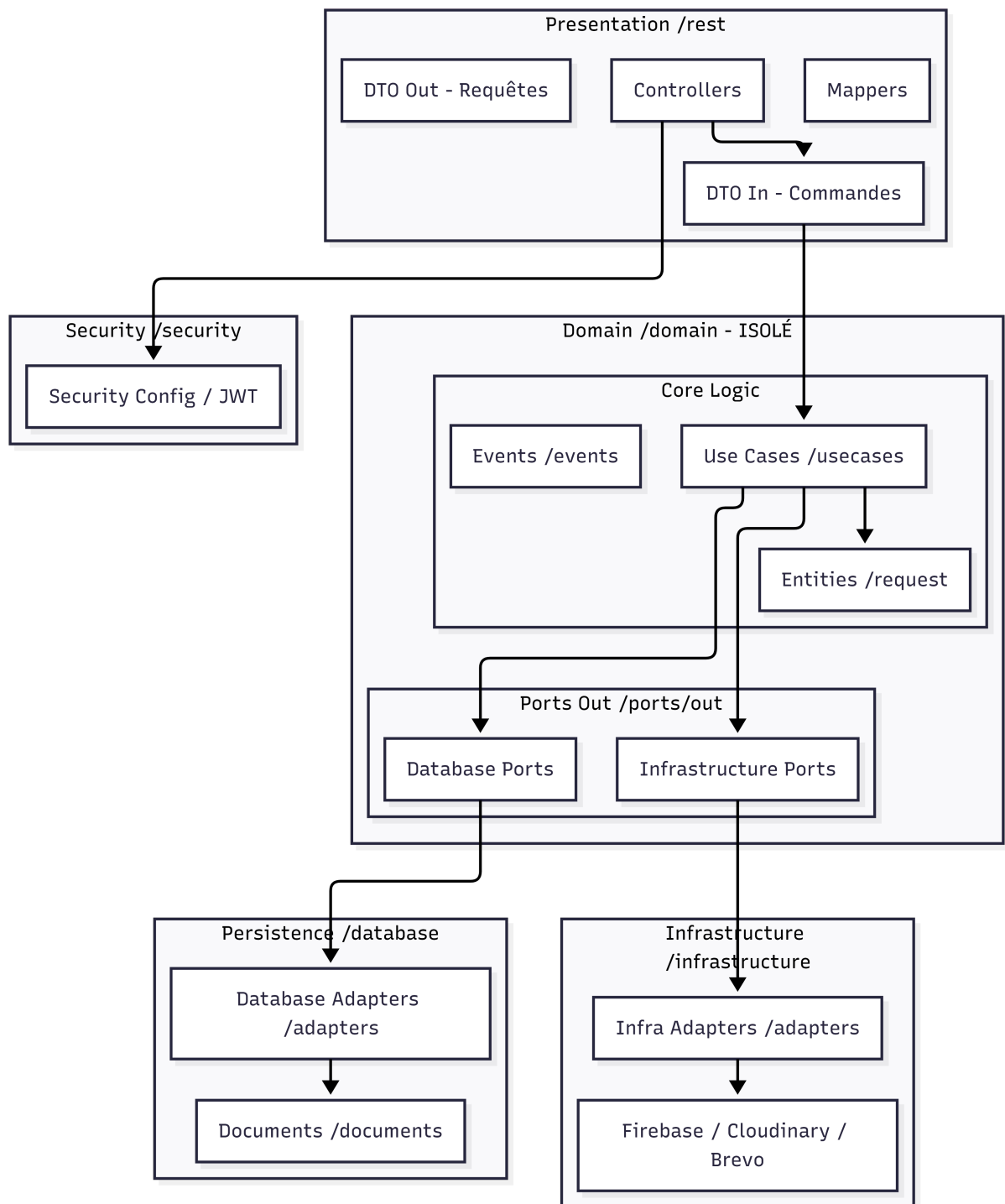


FIGURE 2.2 – Architecture hexagonale du backend

Chapitre 3

Modélisation des Données

Le choix de **MongoDB** comme système de gestion de base de données NoSQL permet une grande flexibilité dans la gestion des signalements, notamment pour le stockage de données hétérogènes (coordonnées GPS, images, métadonnées).

3.1 Schéma de la base de données

L'architecture de la base de données est organisée autour de trois collections principales interconnectées par des identifiants uniques (UUID).

3.1.1 Collection users

Cette collection centralise les informations de profil et les compteurs liés à la gamification.

- **Identification** : `_id` (UUID), `email`, `name`.
- **Sécurité** : `passwordHash`, `authProvider` (Password, Firebase, Google), `role` (CITIZEN, ADMIN).
- **Progression** : `points` (Entier), `badges` (Liste d'objets embarqués).

3.1.2 Collection reports

C'est la collection la plus volumineuse, stockant l'ensemble des interactions citoyennes.

- **Localisation** : `latitude`, `longitude`, `city`, `municipality`.
- **Contenu** : `photoUrl`, `description`, `category`.
- **État** : `status` (En attente, Validé, Rejeté, Résolu).
- **Social** : `reactions` (Liste d'UUID d'utilisateurs), `comments` (Documents embarqués pour limiter les jointures).

3.1.3 Collection badges

Cette collection définit le référentiel des récompenses disponibles dans le système.

- **Identification** : `_id` (UUID), `label` (Nom du badge, ex : "Citoyen Actif").
- **Critères** : `threshold` (Seuil de signalements validés nécessaire pour l'obtention).
- **Description** : `description` (Détail de la récompense ou de l'action accomplie).
- **Média** : `iconUrl` (Lien vers l'image du badge).

3.2 Justification du modèle Document

L'utilisation de documents embarqués (*Embedded Documents*) pour les commentaires et les badges a été privilégiée pour optimiser les performances de lecture. Ce choix permet de récupérer un signalement complet et toutes ses interactions en une seule requête, évitant ainsi des jointures coûteuses et complexes au niveau applicatif.

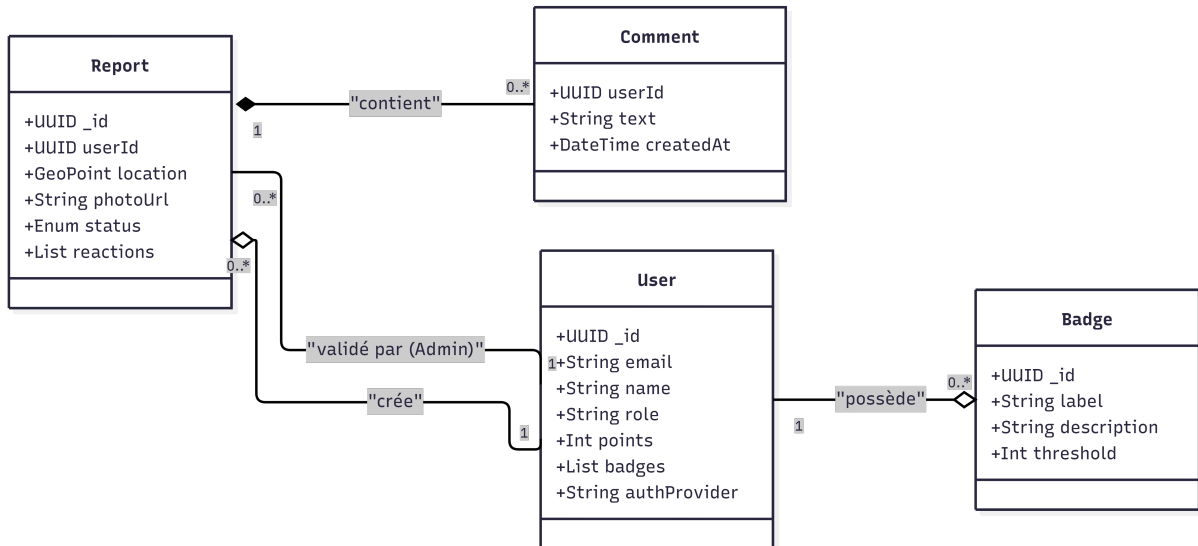


FIGURE 3.1 – Modélisation de la base de données

Chapitre 4

Infrastructure et Déploiement

Ce chapitre décrit l’environnement d’exécution de la solution **Djahiri** et les mécanismes automatisés garantissant la livraison continue du logiciel.

4.1 Stratégie de Containerisation

Pour garantir la portabilité et l’isolement des services, nous avons adopté une stratégie basée sur **Docker**. Chaque composant de l’architecture tourne de manière isolée sur les serveurs.

4.1.1 Isolation des services

Sur chaque serveur (Staging et Production), les conteneurs sont orchestrés via **docker-compose**. Un exemple de l’état des conteneurs sur le serveur de staging montre cette isolation :

- **djahiri-api** : Conteneur Java hébergeant l’API Spring Boot.
- **djahiri-front** : Conteneur Nginx servant les fichiers statiques de l’application ReactJS.

4.1.2 Reverse Proxy et Sécurité

Un serveur **Nginx** installé sur l’hôte agit comme reverse proxy. Il gère :

- La terminaison SSL (HTTPS).
- La redirection des flux vers les bons conteneurs selon le sous-domaine.
- La sécurité des en-têtes HTTP.

4.2 Pipeline CI/CD et Environnements

Le cycle de développement est automatisé grâce à **GitHub Actions**, permettant de réduire les erreurs humaines lors des mises à jour.

4.2.1 Gestion des environnements

Nous maintenons deux environnements distincts sur des VPS Ubuntu isolés :

1. **Staging (Pré-production)** : Utilisé pour valider les nouvelles fonctionnalités avec les bases de données de test sur MongoDB Atlas.
2. **Production** : Environnement final utilisé par les citoyens et administrateurs réels.

4.2.2 Flux de déploiement continu

Le pipeline se déclenche automatiquement selon les branches Git :

- **Build et Test** : Vérification de la compilation et exécution des tests unitaires.
- **Push** : Création et envoi de l'image Docker sur le *GitHub Container Registry* (GHCR).
- **Deploy** : Connexion SSH au VPS et mise à jour automatique des conteneurs via **Ansible** pour garantir un déploiement idempotent.

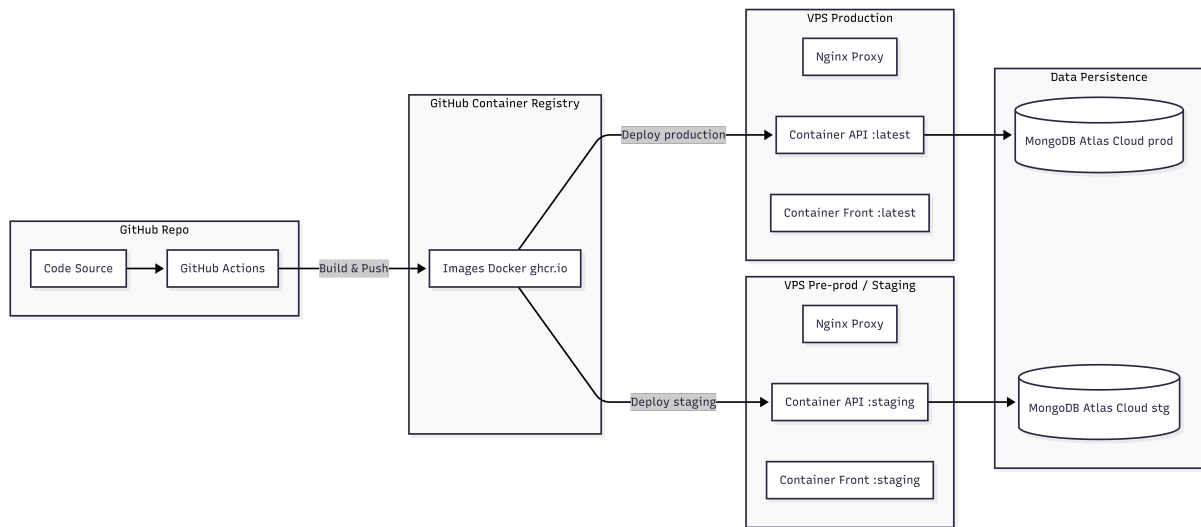


FIGURE 4.1 – Stratégie de déploiement

4.3 Persistance et Cloud

Contrairement aux environnements de développement locaux où MongoDB tourne dans un conteneur Docker, les environnements de staging et de production utilisent **MongoDB Atlas**. Ce choix garantit la haute disponibilité, les sauvegardes automatiques et la sécurité des données citoyennes.